

MONITORING PLAYBOOK

Monitoring Playbook

dealing cluster

CLUSTER dealing cluster

AUDIENCE Platform & on-call engineers

CLASSIFICATION Internal — keep current as the platform evolves

Contents

01	★ Start here: the Overview dashboard	3
02	0. Cluster capacity (so the numbers below mean something)	3
03	1. The dashboards you have, and what each one is for	4
04	2. The 5-minute daily check	4
05	3. Per-dashboard panel reference	6
06	4. Incident workflow: from symptom to root cause	13
07	5. Are these dashboards enough for production RCA? — honest assessment	14
08	6. What to tell the monitoring person — daily checklist (printable)	16
09	7. References	16

Companion to: - [runbook-dealing.md](#) — what to **do** when an alert fires - [cluster-handbook-dealing.md](#) — what each **component** does - [workload-catalog-dealing.md](#) — what's **deployed** where

This doc is for the monitoring person. It says: "here's what each panel measures, here's normal / warning / critical for THIS cluster, here's what to check daily, here's where to start when something looks off."

★ Start here: the Overview dashboard

There is now a single-pane-of-glass dashboard for daily monitoring:

Dashboard URL: [/d/overview-dealing](#) (full URL: <http://10.10.103.203:3000/d/overview-dealing>)

Open this every morning. 4 rows on one screen: - **Row 1 — status gauges (color-coded):** Nodes, Etcd Leader, Running Pods, API avg latency, Cluster CPU %, Cluster Memory % - **Row 2 — control plane trends:** Etcd disk latency + DB size, API request rate by code - **Row 3 — workload signals:** OOM events / restarts / CPU throttling by namespace - **Row 4 — networking:** Cilium drops by reason, HTTP responses by status

If everything is green here, the cluster is healthy. If anything is amber/red, click the linked detail dashboard (top of page) and use §3 to drill into the specific panel.

0. Cluster capacity (so the numbers below mean something)

	Total	Per-node avg
Logical CPUs	30 vCPUs	5 per node (6 nodes)
Memory	~70 GiB	~11.6 GiB per node
Root filesystem	~435 GiB	~72 GiB per node
etcd quota	8 GiB (3 masters)	—
Pod-IP pool	1536 (256/node × 6)	256
Max pods/node	110	—

Thresholds in this document are tuned to those numbers. If the cluster grows or shrinks, re-tune.

1. The dashboards you have, and what each one is for

Dashboard	Folder	Layer	When to open it
Kubernetes / Views / Global	new-kube-cluster	Cluster + node + namespace	First stop for "is the cluster healthy overall?" or "which namespace is using all the CPU?"
Etd by Prometheus	(uncategorized)	Control plane (etcd)	When the cluster feels slow, when alerts about etcd fire, before any RKE2 upgrade
Cilium Metrics	new-kube-cluster	Network agent (per-node)	Network connectivity issues, BPF map pressure, endpoint regeneration spikes
Cilium Operator	new-kube-cluster	Network control plane	IPAM, identity allocation, HA leader election
Hubble Metrics and Monitoring	new-kube-cluster	Network flows + L7 HTTP	"Why is my packet being dropped?", HTTP traffic on Ingress
Hubble L7 HTTP by Workload	new-kube-cluster	(Most panels [N/A])	Skip unless we enable workload-side L7 visibility

How to think about them — the four-layer mental model

```

Layer 1: Cluster control plane    (etcd / apiserver / kube-vip)
      ↓ everything below depends on this
Layer 2: Node infrastructure      (CPU / mem / disk / kubelet / kernel)
      ↓
Layer 3: Workloads + scheduling   (per-pod resources, restarts, scheduler)
      ↓
Layer 4: Network data plane       (Cilium BPF, Hubble flows, Ingress)

```

Rule of thumb: when investigating, start at the layer the user complained about and walk **down**. "My app is slow" → check Layer 3 first (the pod itself), then Layer 2 (the node), then Layer 1.

2. The 5-minute daily check

Run through this once at the start of each day. If anything is red, escalate to the on-call before it becomes a 3 AM page.

A. Cluster-level pass (open **Kubernetes / Views / Global**)

Panel	Normal	Warn	Critical
Nodes	6	5 (one drained planned)	<5

Panel	Normal	Warn	Critical
Namespaces	12 (won't fluctuate)	unusual jump	—
Running Pods	varies	sudden drop >20	—
Cluster CPU Utilization	<40%	60-80%	>85% sustained 10+ min
Global RAM Usage	<60%	75-85%	>90%
CPU Utilization by namespace	platform ns (kube-system, monitoring) under 20%, others scale with apps	a namespace consuming >50% sustained	—
Memory Utilization by namespace	same	same	—
OOM Events by namespace	0	any non-zero in last hour	sustained OOMs
Container Restarts by namespace	0 (steady-state)	1-2 (transient)	restart loop visible

If all green here, the cluster is healthy from a workload perspective.

B. Control-plane pass (open **Etcd by Prometheus**)

Panel	Normal	Warn	Critical
has leader	1 always	drops to 0 ever	drops to 0 sustained → cluster is read-only
db size	<2 GiB	4-6 GiB	>7 GiB (approaching 8 GiB quota)
WAL fsync p99	<30 ms	50-100 ms	>100 ms sustained — apiserver will start slowing down
leader changes (rate)	0	1-2/hour	>5/hour means etcd is flapping
proposals failed total (rate)	0	any non-zero	sustained = quorum issues

C. Networking pass (open **Hubble Metrics and Monitoring**)

Panel	Normal	Warn	Critical
Flows processed Per Node	a few thousand/sec per worker, depends on traffic	sudden drop to 0 on one node	drop to 0 across all nodes
Drop Reasons	mostly empty (steady state); some ct-no-map-found is normal	policy-deny rate non-zero (usually wrong)	sustained drops on any reason

Panel	Normal	Warn	Critical
HTTP Requests (Ingress)	low rate when no users; spikes when users hit UIs	unexpected sustained traffic	—
HTTP Responses by status	200s dominate; some 401 / 304 normal	5xx rate non-zero	5xx rate climbing

That's the 5-minute check. Three dashboards, ~15 panels. If anything is amber/red, drill into the relevant per-dashboard section below.

3. Per-dashboard panel reference

3.1 Kubernetes / Views / Global (uid: `k8s_views_global1`)

The most-used dashboard. Cluster-wide view.

Top stats (numbers at the top)

Panel	What it measures	Normal	When to act
Nodes	<code>count(kube_node_info)</code>	6	Anything <6 = a node is gone. Open §3 IR-3 in runbook.
Namespaces	<code>count(kube_namespace_created)</code>	12	Sudden change = someone created/deleted a namespace (audit log it).
Running Pods	<code>sum(kube_pod_status_phase{phase="Running"})</code>	varies by deploys	Sudden drop = something is evicting/ crashing en masse.

CPU / memory family

Panel	What it measures	Normal	Warn	Critical
Cluster CPU Utilization	(1 - idle) averaged across all 30 vCPUs	<40%	60-80%	>85% sustained 10+ min
Global CPU Usage	bar gauge, more visual than the timeseries	same	same	same
CPU Utilization by namespace	rate of container_cpu_usage_seconds_total grouped by namespace	platform ns <20%	one namespace at 50% sustained	one namespace at 80% — could be a runaway

Panel	What it measures	Normal	Warn	Critical
CPU Utilization by instance	same, per-node	masters higher than workers (apiserver/etcd) — m-1/2/3 maybe 5-10%, workers 1-5% idle	one node above 70% sustained	one node above 85% — node is saturated
CPU Throttled seconds by namespace	sum rate of container_cpu_cfs_throttled_seconds_total	0	rate climbing for any namespace	a namespace consistently throttled = limits too tight
Cluster Memory Utilization	(used / total)	<60%	75-85%	>90% — eviction risk
Global RAM Usage	bar gauge	same	same	same
Memory Utilization by namespace	sum(container_memory_working_set_bytes) by namespace	platform ns <2 GiB, others scale with apps	a namespace climbing without new deploys = leak	sustained climb without limit
Memory Utilization by instance	per-node memory used	spread evenly	one node consistently >80%	one node OOM imminent

Pod state

Panel	What it measures	Normal	When to act
Kubernetes Pods QoS classes	count by Guaranteed/Burstable/BestEffort	mostly Burstable + Guaranteed	spike in BestEffort = someone forgot resource requests (see workload-catalog-dealing.md §2.14)
Kubernetes Pods Status Reason	counts with Status.Reason set (Evicted, NodeLost)	should be 0	any non-zero — something is wrong
OOM Events by namespace	<code>increase(container_oom_events_total[\$_rate_interval])</code>	0	any in last hour = at least one pod killed for memory → check that pod's logs
Container Restarts by namespace	<code>increase(kube_pod_container_status_restarts_total[\$_rate_interval])</code>	0 in steady state; a few during deploys	sustained restarts (CrashLoopBackOff) — IR-4 in runbook

Storage / network

Panel	What it measures	Normal	When to act
Global Network Utilization by device	per-node network bytes/sec	varies; correlates with traffic	sudden sustained spike or drop
Network Received / Transmit by instance	per-node ingress/egress bytes	spread roughly even	one node carrying 5x more = check what's running there
Network Saturation - Packets dropped	<code>node_network_receive_drop_total</code> rate	0	>0 = RX queue saturated; check NIC settings or noisy neighbor
Network Received by namespace	sum(rate(container_network_receive_bytes_total)) by namespace	scales with app traffic	one namespace dominating ingress traffic

Kubernetes Resource Count

Panel	What it shows	Use
Kubernetes Resource Count	small table: deployments, statefulsets, daemonsets, namespaces, etc.	sanity check that nothing has been mass-deleted or mass-created

3.2 Etcd by Prometheus

This dashboard powers the most expensive incidents to debug. Watch it especially around RKE2 upgrades, large deployment rollouts, or when many resources are being created/deleted at once.

Panel	What it measures	Normal	Warn	Critical
etcd up	how many etcd members are reachable	3	2 (still has quorum, fix fast)	1 (cluster read-only)
DB size	<code>etcd_mvcc_db_total_size_in_bytes</code>	<2 GiB steady	growing > 100 MB/day	approaching 8 GiB quota
DB size relative to quota	size / 8 GiB	<30%	70-85%	>85% — defrag immediately (runbook IR-2)
WAL fsync duration	p99 histogram — how long etcd takes to fsync to disk	<30 ms	50-100 ms	>100 ms = disk degrading; apiserver requests will start to fail
Backend commit duration	p99 — disk commit	<25 ms	50 ms	>100 ms

Panel	What it measures	Normal	Warn	Critical
Leader changes	<code>rate(etcd_server_leader_changes_seen_total[5m])</code>	0	1-2/hour	flapping (>5/hour) — network or master health issue
Proposals committed/applied/pending/failed	raft pipeline	pending ≈ 0, failed = 0	failed rate >0	sustained failed
Slow apply/read indexes	rate of slow operations	0	climbing	sustained — apiserver impact
Network peer RTT	round-trip time between etcd members	<5 ms (local LAN)	>50 ms	sustained >100 ms — network issue between masters

Drill-down path: WAL fsync climbing → suspect host disk → ssh master, `iostat -x 1 5`, check `/dev/sdb` (the etcd scsi1 disk).

3.3 Cilium Metrics (uid: `vtuWtdumz`)

Per-node Cilium agent metrics. Use this when: - A node is showing network weirdness - Policy changes are landing - You suspect BPF map exhaustion

Panel	What it measures	Normal	Warn	Critical
CPU Usage per node (cilium-agent)	irate of process_cpu_seconds_total	<5% of one core	20% sustained	>50% sustained — policy churn or BPF overhead
Resident memory	RSS per cilium-agent pod	<500 MiB	1 GiB	approaching the 2 GiB limit (set in chart)
Open file descriptors	sum of FDs	<5000	10000	approaching ulimit
System-wide BPF memory usage	bpf maps + progs virtual memory	varies; should be stable	growing without policy changes	very large — investigate map cardinality
Endpoint state (ready vs regenerating)	count(cilium_endpoint_state) by state	ready ≈ all pods, regenerating ≈ 0	sustained regenerating > 5	regenerating > 20 — policy storm or stuck agent
Endpoint regeneration time — average	_sum/_count of regen seconds	<2 s	5 s	>10 s — agent struggling

Panel	What it measures	Normal	Warn	Critical
Datapath Conntrack Dump Resets	counter of CT-table dump resets	0 (steady state)	rate climbing	indicates heavy connection churn
Cilium API process time	avg API latency	<50 ms	200 ms	>500 ms — agent slow to respond
Errors & Warnings	rate of error/warn log lines, by source and level	~0 (with brief blips during reconciles)	sustained level=error >5/min	sustained level=fatal
Kubernetes events received/processed	counter	rate matches API server activity	drops to 0 = watch broken	sustained 0
Forwarded Traffic / Dropped Traffic (bytes)	sum(rate(cilium_forward_bytes_total)) vs cilium_drop_bytes_total	drop/forward ratio < 0.1%	ratio > 1% sustained	ratio > 10% — something blocking traffic, find which reason in Hubble
Allocated Addresses	cilium_ip_addresses	<1500 (pod CIDR is 1536)	approaching pool limit	full — new pods can't get IPs
# nodes Cilium knows about	cilium_nodes_all_num	6	<6 = a node fell out of the mesh	persistent 5
Policy Apply Latency — average (restored panel)	avg time to enforce a policy	<100 ms	500 ms	>1 s — agent stuck applying
Policy Identity Update Latency — average	avg time per identity update	<50 ms	200 ms	>500 ms

Drill-down path: Errors climbing on one node → `kubectl logs <cilium-pod-on-that-node> -n kube-system` → look for the error keyword → search Cilium GitHub issues.

[N/A] panels in this dashboard (BPF syscall histograms, kvstore, proxy redirects, services) are intentionally hidden — those metrics don't exist on this cluster. See the [N/A] descriptions.

3.4 Cilium Operator (uid: `1GC0TT4Wz`)

Just 2 working panels (the rest are cloud-IPAM panels marked [N/A]). Cilium Operator is mostly idle on this cluster (uses kubernetes IPAM mode, no cloud).

Panel	What it measures	Normal	Watch for
CPU Usage per node	irate of cilium_operator_process_cpu_seconds_total	<2%	sudden spike = identity churn or webhook flapping
Resident memory	RSS of cilium-operator (2 replicas)	~50-100 MiB	unbounded growth = identity leak

3.5 Hubble Metrics and Monitoring (uid: 5HftnJAWz)

The richest networking dashboard for debugging. Where you go when "X can't reach Y."

Flow statistics

Panel	What it measures	Normal	When to look
Flows processed Per Node	rate of all flows handled per cilium-agent (min/avg/max across nodes)	hundreds to thousands per node	drop to 0 on a specific node = that agent is sick
Flows Types	breakdown by <code>type</code> (L3/L4, Trace, L7, etc.)	mostly L3/L4 + Trace, small L7 (from Ingress)	unexpected L7 spike = someone enabled visibility on a workload
Verdict Distribution	FORWARDED vs DROPPED vs ERROR	FORWARDED >99%	DROPPED rising = policy or routing issue

Drop investigation (this is your bread-and-butter)

Panel	What it measures	Normal	Action
Drop Reasons	<code>hubble_drop_total</code> by <code>reason</code>	mostly empty	any sustained non-zero: identify the reason and the source pod (see runbook IR-6)
Drops by source / destination	breakdowns	mostly empty	use this to localize a problem to specific pods/IPs

Most common drop reasons and what they mean:

Reason	Meaning	Where to look
<code>policy-deny</code>	NetworkPolicy/CNP/CCNP denied it	the source pod's policies; <code>kubectl get cnp,netpol -A</code>
<code>ct-no-map-found</code>	conntrack table miss (often benign restart-time noise)	usually self-resolves; investigate if sustained

Reason	Meaning	Where to look
<code>invalid-source-ip</code>	packet with source IP Cilium doesn't recognize	IP spoofing or asymmetric routing
<code>no-route</code>	no route to destination	dest endpoint is gone, or service IP not in BPF map
<code>unauthenticated drop</code>	mTLS / WireGuard handshake issue	WG mesh problem

TCP / ICMP / Protocols

Panel	What it measures	Use
TCP Flags	counts of SYN/FIN/RST/ACK	RST storm = connections aggressively closed; investigate which pods
ICMP	ICMP types observed	unusual ICMP unreachable = something's not reachable
Top 10 Port Distribution	most-used ports	sanity check; sudden new port = new service or scan
Protocol Usage	TCP vs UDP breakdown	mostly TCP for apps; UDP heavy if DNS or quic traffic

L7 HTTP (from Ingress traffic only — see handbook §6.3)

Panel	What it measures	Normal	Watch for
HTTP Requests (Ingress)	requests/sec by method	low when idle, climbs when users hit argocd/hubble UI	sudden traffic from unexpected source
HTTP Responses (Ingress)	by status code	mostly 200 / 304 / 301	5xx rate non-zero = backend in trouble; 401 spike = bad creds being tried
HTTP Latency (avg) by reporter	rate(_sum)/rate(_count)	tens to hundreds of ms	climbing > 1s = backend slow
HTTP Latency (avg) by method	same, broken by method	similar	spot which method is slow (typically POST > GET)
HTTP Protocol Usage	HTTP/1.1 vs HTTP/2 breakdown	mostly HTTP/1.1	shift = client behavior change

DNS panels — [N/A]

These would require enabling Cilium DNS proxy via a CNP with `dns:` rules. Intentionally off on this cluster to avoid latency to app DNS lookups. See cluster-handbook §6.4.

4. Incident workflow: from symptom to root cause

Workflow A — "A user says the app is slow"

1. Open K8s Views Global
 - └ Container Restarts by namespace → are restarts climbing for this app's ns?
 - └ OOM Events by namespace → any OOM for this app?
 - └ CPU Utilization by namespace → is the app CPU-bound?
 - └ Memory Utilization by namespace → is the app memory-bound?
2. If a particular pod is suspicious:


```
kubectl -n <ns> describe pod <pod> | sed -n '/Conditions/,/Events/p'
```

```
kubectl -n <ns> top pod <pod>
```

```
kubectl -n <ns> logs <pod> --tail=200
```
3. Network-layer suspicion (it can reach DB? talk to service?):

Open Hubble Metrics and Monitoring → Drop Reasons → filter by source = your app's namespace

```
Or: kubectl -n kube-system exec <cilium-agent-on-that-node> -c cilium-agent -- hubble observe --from-namespace <app-ns> --verdict DROPPED --last 100
```
4. Is the node bad?

Open K8s Views Global → CPU/Memory by instance → which node hosts the pod?

Check that node's resource panels for pressure.
5. Is the control plane slow?

Open Etcd by Prometheus → WAL fsync p99

If p99 > 100 ms, apiserver is slow, everything is slow.

Workflow B — "Grafana alert: Pod CrashLoopBackOff in production"

1. Identify the pod from the alert annotations.
2. Open K8s Views Global → OOM Events by namespace
 - If non-zero for that ns: the pod was OOM-killed. Check container memory limit vs actual usage. Raise limit or fix the leak.
 - If zero: not memory. Likely app crash or probe failure.
3.

```
kubectl -n <ns> describe pod <pod>
```

 → look at "Last State" reason.

```
kubectl -n <ns> logs <pod> --previous
```

 → app's last words.
4. Probe failure? Open the `prober_probe_total` in Mimir (query the metric directly, no panel for this yet – TODO add).
5. Image pull failure? `kubectl describe` shows it. Check registry connectivity from a debug pod on the same node.

Workflow C — "Service X can't reach Service Y"

This is the textbook case. The methodology is [runbook-dealing.md](#) IR-6 (Hubble drops first):

1. Open Hubble Metrics and Monitoring → Drop Reasons. If `policy-deny` is non-zero:
→ Open the source pod's NetworkPolicy/CNP/CCNP. Likely fix is to extend the egress allowance to the destination.
2. If drops show `no-route`:
→ Is the destination Service IP in BPF? Use `cilium-dbg bpf lb list` on the source node.
→ Is the destination pod actually there? `kubectl get pods -A | grep <name>`.
3. If no drops are visible but the connection still hangs:
→ It's the encrypted-host-IP path bug we hit on 2026-05-20. See runbook §IR-6.
→ `kubectl -n kube-system exec <agent> -c cilium-agent -- hubble observe --from-pod <src-ns>/<src-pod> --last 100` (no verdict filter – looking for absence)
4. TLS error: source side initiates handshake, dest side rejects:
→ cert problem (see runbook IR-7) or wrong port.

5. Are these dashboards enough for production RCA? — honest assessment

What you CAN diagnose from the current setup

Question	Yes/ no	How
Is the cluster API healthy?	✓	<code>/readyz</code> , etcd dashboard
Is a node degrading?	✓	K8s Views Global per-instance + node_cpu/memory panels
Did a pod crash, and why?	✓	OOM panel + restart panel + <code>kubectl logs --previous</code>
Is a namespace hogging resources?	✓	CPU/Memory by namespace
Is a pod CPU-throttled?	✓	CPU Throttled seconds by namespace
Is a packet being dropped by policy?	✓	Hubble Drop Reasons + <code>hubble observe --verdict DROPPED</code>
Is etcd slow?	✓	Etcd by Prometheus → WAL fsync p99
Is Prometheus losing data?	✓	<code>prometheus_remote_storage_samples_failed_total</code> (query directly)

What you CANNOT diagnose without adding more

Question	Why not	What it'd take
What URL did the user hit when they got a 500?	We capture HTTP metrics at the Envoy/Ingress layer, but not per-URL/per-user; no request logging in Mimir	Send Envoy access logs to Loki; correlate with metrics by trace-id
Which database query made the app slow?	We don't have app-level metrics or query traces	App-level instrumentation (OpenTelemetry / app-emitted Prometheus metrics)
Did all 3 microservice hops complete? Which was slow?	Need distributed tracing	Add Grafana Tempo and instrument apps with OpenTelemetry SDKs
Why is DNS slow inside pods?	hubble_dns_* not flowing (workload L7 visibility off)	Enable CNP with <code>dns:</code> rules — adds latency, decision is yours
What's the trend over 3 months on this metric?	Mimir retention determines this; we don't know the retention policy on <code>mimir.stackflow.org</code>	Confirm Mimir retention; if it's <90 days, may need to extend
Was a specific Pod evicted, and why?	Eviction shows in OOM panel; reason details in <code>kubectl describe node</code> events	More detail via PrometheusRule <code>KubeNodeUnreachable</code> etc.
Did Pod X talk to Pod Y at a specific past time?	Hubble flow logs go to Loki; queryable but no preset dashboard	Build a Hubble flow query in Loki Explore

Recommended additions to reach "comprehensive production RCA"

Listed by ROI:

1. **An "Overview" / single-pane-of-glass dashboard.** One screen with the 12 most-important panels from all the dashboards above — what the monitoring person opens first every morning. (Could write this — ~1 hour.)
2. **Probe metrics panel** — `prober_probe_total{result="failed"}` is flowing but not on a dashboard. Add a row in K8s Views Global. (Could write this — 20 minutes.)
3. **Alert routing inventory.** When a Grafana alert fires to Teams, where does the human go to investigate? Document the 5-10 critical Grafana alerts with their corresponding dashboard + IR pointer. (Could write — ~1 hour.)
4. **App-level metrics convention.** Decide that every app deployed exposes a `/metrics` endpoint with at least: request count by status, request latency histogram. Add a ServiceMonitor template. (Architecture decision + ~2 hours implementation per app.)
5. **Distributed tracing** (Grafana Tempo). Major undertaking; only worth it once you have ≥3 microservices that talk to each other.

6. What to tell the monitoring person — daily checklist (printable)

Print this and put it next to the desk.

DAILY CHECK (5 minutes)

- Open K8s Views Global
 - Nodes = 6 (red: anything less)
 - Cluster CPU < 70% (yellow at 70%, red at 85%)
 - Cluster Memory < 80%
 - OOM Events by namespace = 0 in last hour
 - Container Restarts by namespace = 0 (or low and not climbing)
- Open Etcd by Prometheus
 - etcd has-leader = 1 (red: 0)
 - DB size < 6 GiB
 - WAL fsync p99 < 50 ms (red: > 100 ms)
- Open Hubble Metrics and Monitoring
 - Flows processed Per Node not at 0 on any node
 - Drop Reasons: no `policy-deny` rate climbing
 - HTTP Responses: 5xx rate near 0

ANYTHING RED OR AMBER → notify on-call IMMEDIATELY
 ANYTHING UNUSUAL BUT GREEN → mention in standup

WEEKLY CHECK (15 minutes; do once a week)

- Open Cilium Metrics → Endpoint state → all `ready`
- Open Cilium Metrics → Errors & Warnings → no sustained level=error
- Open K8s Views Global → Network Saturation - Packets dropped → all 0
- Open Etcd by Prometheus → Leader changes rate → near 0
- Open Cilium Operator → memory of cilium-operator stable, not growing

If you see any of these go non-zero or trend up:
 → Tell on-call → open runbook-dealing.md

7. References

- [runbook-dealing.md](#) — what to **do** when an alert fires (8 IRs)
- [cluster-handbook-dealing.md](#) — what each component **is** (24 sections + port inventory)
- [workload-catalog-dealing.md](#) — what's **deployed** where
- [policy-reviewer.md](#) — checklist for **adding** a policy
- [troubleshooting.md](#) — generic landing page across docs
- [Cilium Metrics Reference](#)
- [Hubble Metrics Reference](#)
- [etcd alerting playbook](#)

Update this doc whenever: - A new dashboard is added → document its purpose + key panels here - A panel's normal range shifts because cluster size changed → re-tune thresholds - A new monitoring person joins → walk them through this in person on day 1